

# **Celebrity Face Recognition Project**

This project implements a classical machine learning pipeline for celebrity face recognition using Python, OpenCV, Scikit-learn, and Flask, covering the full workflow from data collection to model deployment. Here's an in-depth breakdown and insights on how to explain it in interviews, plus answers for typical interview questions.

## **Project Workflow and Explanation**

### **1. Face and Eye Detection**

- Utilized OpenCV's Haar cascade classifiers to detect faces and eyes in input images, a robust method for frontal face detection using feature-based approaches.
- Converted images to grayscale for faster and more accurate detection.
- Drew bounding boxes around detected faces and eyes for visualization and preprocessing.

### **2. Data Preprocessing and Cleaning**

- Cropped detected face regions and filtered out poor detections using custom logic, ensuring only high-quality face data entered your model.
- Applied wavelet transforms (using PyWavelets) for feature extraction, capturing both spatial and frequency characteristics of the face images. This combines raw pixel and transformed features to enrich the dataset.
- Visualized preprocessed samples using matplotlib for manual assessment and validation.

### **3. Dataset Construction and Organization**

- Automated dataset construction: Scraped images, used OpenCV for cropping, stored images in class-specific directories based on celebrity names.
- Maintained dictionaries mapping images to classes for easy training and inference.

### **4. Feature Extraction**

- Used raw, resized image arrays and their wavelet representations (Haar or Daubechies) as features, vertically stacking these for each sample, resulting in robust input vectors.

## 5. Model Training and Optimization

- Tried multiple classical ML models—SVM (RBF), Random Forest, Logistic Regression—with hyperparameter tuning (GridSearchCV) wrapped in scikit-learn pipelines for scaling and training.
- Used training/test splits for evaluation.
- Scored models using accuracy and classification report metrics.

## 6. Model Serialization

- Exported the best model as a pickle (.pkl) using joblib for later reuse/deployment.
- Saved the class dictionary mapping for future inference.

## Explaining in Interviews

- State that you used classical computer vision and ML (not deep learning) for celebrity classification due to limited data and interpretability.
- Emphasize data preprocessing, feature engineering (wavelets), and classical pipelines—this shows depth in ML knowledge, crucial for non-DL/edge devices.
- Discuss challenges: noisy detections, poor-quality images, need for manual validation, and solutions like cascading checks for face/eye and visualization.
- Mention modular design—separate data, processing, ML, API layers (good for scalability).

## Common Interview Questions and Sample Answers

| Question                                   | Answer                                                                                                                                                                                |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| How does Haar Cascade face detection work? | Haar cascades use rectangular features and an integral image for fast computation, applying classifiers trained on positive (face) and negative (non-face) samples to localize faces. |

|                                                          |                                                                                                                                                                                       |
|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why use wavelet transforms for face features?            | Wavelets efficiently extract locality-based frequency features, capturing texture details not visible via raw pixels, making the model robust to brightness and alignment variations. |
| Why not use deep learning for this?                      | Classical approaches are interpretable, faster on CPU, and require less data—ideal for limited celebrity samples and real-time requirements.                                          |
| How do you handle poor detections or misclassifications? | Implemented cropping only when detection passes quality checks (typically requiring two eyes), used manual inspection, and ensemble predictions for reliability.                      |
| How is your pipeline modular?                            | Separated UI (HTML/JS), backend (Flask API), image processing, and ML model inference. Each in its function for maintainability and testing.                                          |
| What metrics did you use to evaluate performance?        | Used accuracy, F1-score, confusion matrix, and classification report, crucial for multi-class settings where class imbalance may occur.                                               |

## Additional Technical Insights

- OpenCV's haarcascades are robust but not infallible—consider lighting, pose, and occlusions as limitations.
- Wavelet transforms (e.g., Daubechies) can highlight the “edges” and fine-grained features compared to simple resizing—this can be explained with examples for images with defined jawlines or glasses.
- Scikit-learn provides a handy pipeline for preprocessing and SVM training—GridSearchCV automates hyperparameter tuning.

- Saving models and associated metadata is essential for deploying on new data or serving via APIs.

Summary:

- This Flask server provides an API endpoint `/predict` that accepts a base64 encoded image via a POST request.
- It decodes and preprocesses the image by resizing and applying a wavelet transform using your custom `w2d` function.
- The preprocessed image features are fed into the pre-trained machine learning model to predict the celebrity class.
- The API returns the predicted class label, the highest probability, and the probabilities for all classes as a JSON response.

This modular API design enables integration with web frontends or other clients for real-time celebrity face classification.

Here is a summary and explanation of the frontend files of your celebrity face recognition project:

## **app.html**

- The main user interface HTML that provides an image upload form.
- Contains an input field for browsing images and a button to submit for classification.
- Div containers to display results returned from the server, including predicted celebrity name and confidence.
- Linked CSS (`app.css`) and JavaScript (`app.js`) for styling and dynamic functionality.

## **app.css**

- Styles the webpage layout with modern UI components.
- Uses flexbox/grid for image upload area arrangement.
- Styles buttons, inputs, and result display sections for clean user experience.
- Provides responsiveness and consistent spacing, fonts, and colors for a polished design.

## **app.js**

- Handles image uploads, converts images to base64 strings.
- Sends AJAX POST requests to your Flask API `/predict` endpoint with the base64 encoded image.

- Processes JSON response from the server to display prediction results dynamically.
  - Manages UI states (loading indicators, error handling, resetting display on new uploads).
  - Implements event listeners and modular functions for maintainable code.
- 

## How to explain in an interview

- The frontend is a simple yet efficient React-less web app for quick celebrity face predictions.
- It relies on AJAX requests to communicate asynchronously with the server, showing results without page reloads.
- The file upload flow involves converting images to base64, sent to the backend—demonstrating your understanding of client-server data exchange.
- Styling is clear and intuitive, using CSS flexbox/grid, which highlights your frontend basics competency.
- The JavaScript code is modular with clear functional separation for maintenance and extensibility.

## Project Summary

### Overview

A complete machine learning-based celebrity face recognition system that uses classical computer vision techniques and traditional ML algorithms to classify faces of five famous sports personalities. It includes data preprocessing, model training, evaluation, and full-stack deployment with a Flask backend and a web frontend.

## Key Components

### 1. Face and Eye Detection (Preprocessing)

- Used OpenCV Haar cascade classifiers for detecting faces and eyes in images.
- Cropped face regions containing two eyes to ensure quality training data.
- Performed manual and automated data cleaning and checked image existence.
- Applied wavelet transforms (PyWavelets) to enrich raw pixel data with texture features.

## 2. Dataset Preparation

- Scraped celebrity images from Google.
- Cropped detected faces and organized images per celebrity class.
- Created mappings of image files to labels for machine learning.

## 3. Feature Extraction & Model Training

- Combined resized raw images and wavelet features into single input vectors.
- Trained multiple models (SVM, Random Forest, Logistic Regression) with hyperparameter tuning using GridSearchCV.
- Evaluated models using accuracy, classification report, and confusion matrices.
- Saved best-performing models and class dictionaries for deployment.

## 4. Backend Server (Flask API)

- Provided `/predict` endpoint accepting base64 encoded images.
- Preprocessed uploaded images similarly as training (resize + wavelets).
- Used the trained model to predict celebrity classes and return confidence scores.
- Designed API to send JSON responses of predictions and probabilities.

## 5. Frontend (HTML, CSS, JavaScript)

- Uploaded images via a form and converted them to base64 to send to backend.
  - Received and dynamically displayed prediction results without page reload.
  - Used CSS Flexbox/Grid for clean and responsive design.
  - Managed UI states (loading, result display, errors) with modular JavaScript code.
- 

## Interview Preparation

### How to Explain Your Project

- Briefly introduce the goal: A classical ML-based celebrity face recognition system using image preprocessing, feature engineering through wavelets, and traditional classifiers.
- Explain the rationale for classical CV vs. deep learning—interpretability, data size, and computational efficiency.

- Describe data pipeline: scraping → face/eye detection → cropping with quality checks → feature extraction (pixels + wavelets).
- Discuss model selection, tuning, and evaluation—importance of pipelines, scaling, grid search, and metrics.
- Walk through your full-stack integration: serving predictions via Flask, front-end interaction with AJAX.

## Common Interview Questions and Suggested Answers

| Question                                                       | Answer                                                                                                                                                  |
|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why use Haar cascades for face detection?                      | Haar cascades are efficient, well-tested, work well for frontal faces, and do not require GPUs, suitable for smaller datasets and real-time processing. |
| What are wavelet transforms and why use them?                  | Wavelets capture local frequency / texture information in images, improving model robustness beyond raw pixels against noise and lighting changes.      |
| Why did you combine raw pixels and wavelets for features?      | Combining preserves color and structural info alongside texture details for better classification.                                                      |
| Why traditional ML classifiers instead of deep learning?       | Classical methods require less data, are easier to train/debug, and perform well with engineered features on small datasets.                            |
| How did you handle false face detections or poor crop quality? | Detected faces were only kept if two eyes were found, otherwise discarded. Manual inspection was done to ensure dataset quality.                        |

|                                                                    |                                                                                                                                                          |
|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| How does the Flask API interact with the frontend?                 | Frontend sends base64-encoded images via AJAX POST requests; backend preprocesses and predicts using saved model; frontend displays results dynamically. |
| What metrics did you use to evaluate model performance?            | Accuracy, classification reports (precision, recall, F1-score), confusion matrices to analyze multi-class performance comprehensively.                   |
| How did you optimize hyperparameters?                              | Used GridSearchCV with cross-validation inside scikit-learn pipelines, tuning kernel types and regularization in SVM, tree counts in Random Forest, etc. |
| Explain the role of the preprocessing function in the server code. | It decodes base64 image, resizes, applies wavelet transform, stacks features into a vector suitable for the input model.                                 |

## Tips for the Interview

- Be ready to discuss tradeoffs: classical CV/modeling vs. modern deep learning.
- Understand the math/intuition behind Haar cascades and wavelets.
- Highlight full-stack capabilities showing you didn't just do modeling but a deployable system.
- Discuss scaling possibilities and limitations.
- Prepare to write or explain code snippets—face detection, feature stacking, API request handling.